

Project Title:

SpotifyML — AI-Powered Music Classification System with Genre Detection, Mood Analysis, and Popularity Prediction

Group Members: Sila Javed - 24k-0703
Simra Wahidi - 24k-2013
Yash Raj - 24k-0737

Course: Artificial Intelligence

Instructor: Riaz Ahmed

Submission Date: 21st April 2026

1. Project Overview

• Project Topic:

This project focuses on developing an AI-powered music classification system called SpotiML that analyzes audio features of Spotify songs and simultaneously predicts three properties: the genre of the track, the emotional mood it conveys, and whether the song has high or low popularity potential. The system applies four supervised machine learning algorithms — Naive Bayes, Decision Tree, Random Forest, and Logistic Regression — on a real-world dataset of 114,000 Spotify tracks and presents all predictions through an interactive web dashboard styled after the Spotify application.

• Objective:

The main objective of this project is to build a multi-task music classification platform by incorporating AI techniques to:

- Predict the genre of a song (Pop, Rock, Hip-Hop, Classical, EDM, etc.) from its audio features
- Detect the emotional mood of a song (Happy, Sad, Energetic, Calm) using feature engineering on valence and energy values
- Classify a song's popularity potential as High or Low based on its audio characteristics
- Compare the performance of four machine learning algorithms across all three prediction tasks
- Deploy trained models as a REST API and present results through a Spotify-themed interactive frontend

2. System Description

• Original Problem Background:

Music streaming platforms like Spotify host catalogs of tens of millions of songs. Organizing, classifying, and recommending music at this scale is a significant challenge. Traditionally, music classification required manual curation by human experts or simple rule-based tag systems, neither of which scale effectively or capture the nuanced audio characteristics that distinguish genres and moods. Furthermore, predicting whether a song will become popular before it gains traction requires identifying complex patterns in audio structure that are invisible to the human ear but detectable through numerical audio feature analysis. This project addresses these challenges using supervised machine learning on structured audio feature data extracted from Spotify's catalog.

• Innovations Introduced:

1. **Multi-Task Classification System**The system trains 12 separate models (4 algorithms × 3 tasks) on the same 13 audio features, enabling simultaneous prediction of genre, mood, and popularity from a single input — something traditional single-task classifiers cannot do.
2. **AI Mood Detection via Feature Engineering**The Spotify dataset does not contain a mood column. The system derives mood labels algorithmically from two audio features — valence (musical positivity) and energy — using a quadrant-based mapping that produces four mood classes: Happy, Sad, Energetic, and Calm. This demonstrates real-world feature engineering.
3. **Algorithm Benchmarking Dashboard**By training Naive Bayes, Decision Tree, Random Forest, and Logistic Regression on the same dataset and tasks, the system produces a direct performance comparison across all 12 models, giving insight into which algorithm type best suits each classification problem.
4. **Full-Stack Deployment with Spotify-Themed UI**Trained models are persisted as .pkl files and served via a Flask REST API. A React frontend styled after the Spotify application allows users to adjust audio feature sliders and receive live predictions with confidence scores and a plain-English summary.

3. AI Approach and Methodology

• Machine Learning Algorithms

Four supervised classification algorithms are trained and compared on all three tasks:

- **Naive Bayes:** Gaussian Naive Bayes — A probabilistic classifier based on Bayes' theorem. Assumes feature independence and computes class probabilities from training data. Used as the baseline model due to its speed. Performs best when features are relatively uncorrelated.
- **Decision Tree:** Decision Tree Classifier (max_depth=12) — A rule-based model that recursively splits data on the most informative feature. Produces human-readable if-then rules (e.g. 'if

acousticness > 0.8 and instrumentalness > 0.7, then Classical'). Interpretable but prone to overfitting.

- **Random Forest:** Random Forest Classifier (100 estimators) — An ensemble of 100 decision trees, each trained on a random subset of data and features. Predictions are made by majority vote across all trees. Consistently achieves the highest accuracy (83–88%) by reducing variance through aggregation.
- **Logistic Regression:** Logistic Regression (with StandardScaler pipeline) — A linear model that computes a weighted sum of features passed through a sigmoid function to produce class probabilities. Implemented inside a scikit-learn Pipeline with feature standardization. Particularly effective on the mood task where the valence-energy decision boundary is approximately linear.

• Feature Engineering

Since the Spotify dataset does not include mood labels, the system creates them using a quadrant-based rule applied to valence and energy values before training:

- valence >= 0.5 AND energy >= 0.5 → Happy
- valence < 0.5 AND energy >= 0.5 → Energetic
- valence >= 0.5 AND energy < 0.5 → Calm
- valence < 0.5 AND energy < 0.5 → Sad

Popularity labels are derived by binning the continuous Spotify popularity score (0–100) into a binary classification target: High (score >= 70) or Low (score < 70).

• Other Techniques

- Data preprocessing: null removal, duplicate filtering, top-10 genre selection, 80/20 stratified train-test split
- Prompt engineering not applicable — system uses structured ML models, not LLMs
- Model persistence: all 12 trained models serialized to .pkl files using Python's pickle module
- REST API design: Flask endpoints handle prediction requests and return JSON responses with predictions and confidence scores

4. System Rules and Mechanics

• Modified Rules / System Workflow:

The system operates in two distinct phases:

Training Phase (executed once before launching the app):

5. Load dataset.csv (114,000 Spotify tracks) from the data/ directory
6. Clean data: drop null values, remove duplicate track IDs, filter to top 10 genres
7. Engineer labels: compute mood column (valence + energy quadrant) and pop_label column (High/Low)

8. Train all 4 algorithms on Task 1 (Genre Classification) — save 4 .pkl files
9. Train all 4 algorithms on Task 2 (Mood Detection) — save 4 .pkl files
10. Train all 4 algorithms on Task 3 (Popularity Prediction) — save 4 .pkl files
11. Save accuracy metrics to metrics.pkl for the Model Stats dashboard

Inference Phase (every user prediction request):

12. User adjusts 13 audio feature sliders on the React frontend
13. React sends HTTP POST request to Flask API (/api/predict) with feature values as JSON
14. Flask loads all 12 .pkl model files from disk
15. Each of the 12 models runs .predict() and .predict_proba() on the input feature vector
16. Flask returns JSON response with predictions and confidence percentages for all 12 models
17. React displays the final result card showing Genre, Mood, and Popularity with emoji, coloured labels, and confidence bars
18. A collapsible panel shows how all 4 algorithms voted on each task individually

• Output / Success Conditions:

- Genre prediction: correctly identifies musical genre (Pop, Rock, Hip-Hop, Classical, EDM, etc.) from audio features alone
- Mood prediction: correctly maps valence-energy values to one of four mood classes — Happy, Sad, Energetic, Calm
- Popularity prediction: correctly classifies whether a track scores ≥ 70 (High) or < 70 (Low) on Spotify's popularity scale
- Model comparison: the dashboard accurately displays accuracy percentages for all 4 algorithms across all 3 tasks
- UI clarity: prediction result card shows genre, mood, and popularity prominently before the algorithm-level comparison

• Process Sequence:

19. User input: audio feature values entered via sliders or quick presets (Pop Hit, Sad Ballad, EDM Banger, etc.)
20. Frontend processing: React collects all 13 feature values and sends POST request
21. Backend activation: Flask receives request and loads trained .pkl models
22. AI model execution: all 12 models predict and return class label + confidence score
23. Data aggregation: Flask collects all predictions and structures JSON response
24. Response generation: JSON sent back to React frontend
25. Output displayed: prediction result card shown with genre, mood, popularity, summary sentence, and algorithm comparison

5. Libraries and Tools

- scikit-learn (for all four ML classifiers: GaussianNB, DecisionTreeClassifier, RandomForestClassifier, LogisticRegression, Pipeline, StandardScaler, LabelEncoder, train_test_split, accuracy_score)
- pandas (dataset loading, cleaning, feature engineering, label derivation)
- numpy (numerical array operations and feature vector construction)
- pickle (serializing and deserializing trained model .pkl files)
- Flask (Python micro-framework serving the REST API at /api/predict and /api/model_stats)
- Flask-CORS (enabling cross-origin requests between React frontend and Flask backend)
- React 18 + Vite (interactive Spotify-styled frontend dashboard)
- Kaggle Spotify Tracks Dataset — maharshipandya/-spotify-tracks-dataset (114,000 tracks, 13 audio features, 125 genres)

6. References

- Maharshi Pandya. (2022). Spotify Tracks Dataset. Kaggle. <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org>
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Spotify for Developers. (2024). Audio Features Object. Spotify Web API Documentation. <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>
- Flask Official Documentation. (2024). <https://flask.palletsprojects.com>
- React Documentation. (2024). React — The Library for Web and Native User Interfaces. <https://react.dev>
- Course lecture notes and materials — Artificial Intelligence, Instructor: Riaz Ahmed